INTRODUCCIÓN

El documento presente tiene por finalidad el mostrar el desarrollo de un sistema de asistencia a la conducción para distintas situaciones dentro de la ciudad.

El objetivo de este proyecto es aplicar los conocimientos adquiridos en la materia de Visión y Animación por computadora durante el presente semestre (Primavera 2025). Así mismo el poder adquirir intuición en el uso de estas tecnologías para resolver problemas relevantes en nuestra sociedad.

A través del análisis de imágenes y secuencias de video, los sistemas de visión por computadora pueden extraer información útil, como la detección de objetos, el seguimiento de movimiento, el reconocimiento de patrones y la segmentación de escenas. Por otro lado, la Animación por Computadora complementa permite la representación visual dinámica y realista de escenas, modelos o eventos, lo cual resulta fundamental en simulaciones, visualización de datos o generación de contenido gráfico interactivo.

Estas tecnologías se componen de múltiples áreas de estudio como procesamiento digital de imágenes, geometría computacional, aprendizaje automático y gráficos computacionales. Su integración se ha convertido en una herramienta esencial en sectores como la industria automotriz, medicina, vigilancia, agricultura inteligente, manufactura automatizada y entretenimiento digital.

Descripción del proyecto

El proyecto consiste en el desarrollo de un sistema de asistencia a la conducción enfocado en situaciones comunes dentro de entornos urbanos. A través del uso de técnicas de visión por computadora, el sistema es capaz de identificar elementos relevantes en la vía como carriles de circulación, vehículos cercanos, obstáculos y esquinas o intersecciones que podrían representar un cambio en la trayectoria o riesgo potencial. El sistema está diseñado para operar en tiempo real utilizando video como fuente de entrada, ya sea desde una cámara externa o pregrabada, y busca simular el comportamiento de una herramienta de apoyo para vehículos semiautónomos.

La arquitectura del sistema está dividida en distintos módulos especializados: detección de carriles, detección de objetos mediante redes neuronales, estimación de distancias, especificación de histograma para mejorar visibilidad, realce de colores característicos en la vía y un módulo adicional de detección de esquinas para anticipar posibles entradas o cruces. Todo esto se integra en una única aplicación capaz de visualizar los resultados procesados en tiempo real.

Importancia del Proyecto

Los accidentes de tráfico representan una de las principales causas de muerte y lesiones en todo el mundo. Según datos de la Organización Mundial de la Salud, más de 1,3 millones de personas mueren cada año debido a accidentes de tráfico, y entre el 80% y el 90% de estos incidentes son atribuibles a errores humanos .

En México, se estima que anualmente ocurren alrededor de 24,000 muertes relacionadas con accidentes viales, con un costo asociado de aproximadamente 126,000 millones de pesos al gobierno federal . Este panorama resalta la necesidad urgente de implementar soluciones tecnológicas que asistan al conductor y minimicen los riesgos en las vías. Más aún, si tomamos en cuenta que las redes de carreteras del país no están en las mejores condiciones.

La relevancia de este proyecto subyace en que se aborda uno de los retos más actuales en la industria automotriz: el desarrollo de tecnologías inteligentes para mejorar la seguridad y la autonomía en la conducción. Los conductores se enfrentan constantemente a situaciones complejas como carriles poco visibles, peatones, vehículos mal estacionados o intersecciones poco señalizadas.

Al desarrollar un sistema accesible y efectivo, que pueda interpretar esta información visual y brindar apoyo en la toma de decisiones contribuye significativamente a reducir el riesgo de accidentes y mejorar la eficiencia del tránsito. Se contribuye a la reducción de accidentes y se promueve una cultura de conducción más responsable y consciente.

Además, este proyecto pone en práctica tecnologías que están en el núcleo de muchas innovaciones contemporáneas: la visión por computadora, el aprendizaje profundo y el

procesamiento inteligente de imágenes. La implementación de estas técnicas en un contexto real permite no solo reforzar el aprendizaje académico, sino también aportar soluciones tecnológicas aplicables en la vida cotidiana.

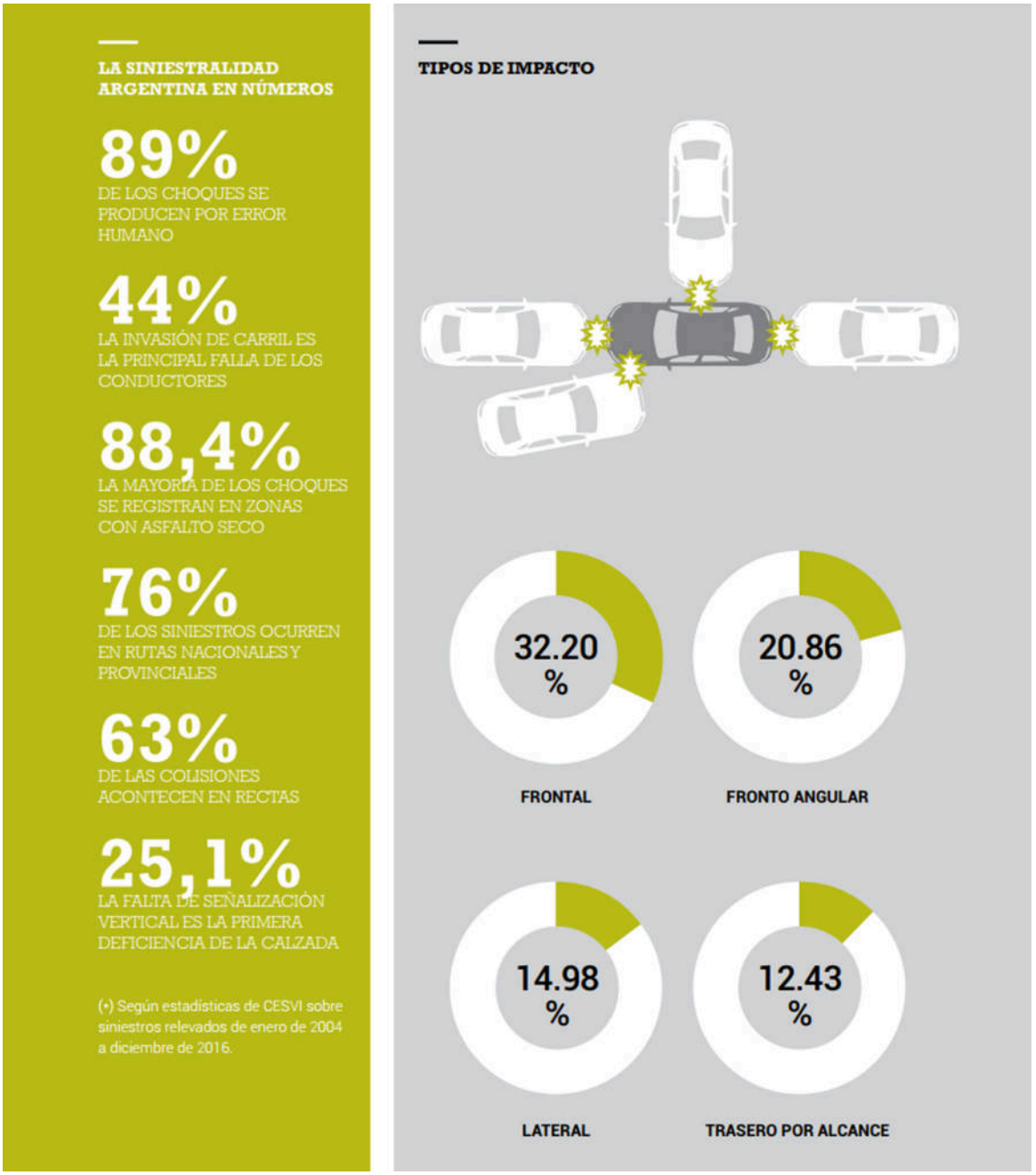


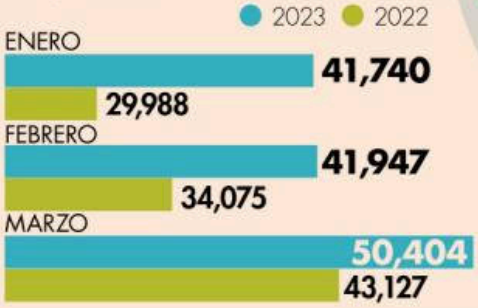
Imagen 1

Accidentes de auto, causa de empobrecimiento en México

Durante el año pasado la cantidad de accidentes viales donde los automóviles contaban con un seguro fue de 487,522, según cifras de la AMIS.



Accidentes viales | I TRIM. DE CADA AÑO, UNIDADES



Los cinco accidentes viales más comunes

1. **Alcance.** Para evitarlos hay que guardar siempre distancia con respecto a otros vehículos, así como no conducir distraído o a exceso de velocidad, pues, la capacidad de reaccionar al frenar será clave.
2. **Cambio de carril.** Sucede cuando un auto quiere rebasar a otros, acción peligrosa en particular en carreteras de doble sentido. Por ello, es importante realizar los cambios de carril por el lado correcto.
3. **Reversa.** Un mal movimiento puede causar un choque laminero, por esto, es indispensable tomarse el tiempo suficiente para ajustar los espejos y ver que no haya otros carros o peatones detrás.
4. **Corte de circulación.** Al ser ocasionado por otros vehículos, este tipo de incidentes escapa a tu control, sin embargo, es recomendable que guardes tu distancia y estés atento ante posibles señalizaciones.
5. **Contra objeto fijo.** Además de lo anterior, se debe tener buena visibilidad en las cuatro direcciones y a larga distancia, especialmente al conducir en condiciones adversas como lluvia o niebla.

FUENTE: ZURICH Y AMIS

EL ECONOMISTA

Imagen 2: Recuperada de el economista.



Imagen 3: Principales causas de accidentes viales.

MATERIALES Y METODOLOGÍA

Para el desarrollo de nuestro proyecto hicimos uso de los siguientes recursos:

- Python
- Anaconda
- Open CV
- Numpy
- Tarjeta gráfica (con drivers actualizados y CUDA)

El desarrollo del presente proyecto se llevó a cabo mediante una metodología basada en etapas. A continuación, se describen las fases principales:

1. ***Análisis del Problema y Definición de Requerimientos***

Se identificó la necesidad de mejorar la percepción visual de un vehículo en entornos urbanos, específicamente en tareas como detección de carriles, obstáculos, distancias y esquinas relevantes.

2. Diseño del Sistema

Se planteó la arquitectura modular del sistema, dividiéndolo en componentes independientes: preprocesamiento de imágenes, detección de carriles, identificación de obstáculos mediante modelos de deep learning (YOLO), estimación de distancias y detección de esquinas. Esta modularidad facilitó el proceso de depuración y permitio ajustar o reemplazar partes específicas sin afectar al resto del sistema.

3. Selección y Preparación de Herramientas

Se seleccionaron herramientas y bibliotecas de visión por computadora (como OpenCV), inteligencia artificial (Ultralytics YOLO) y procesamiento numérico (NumPy), junto con el lenguaje de programación Python. También se implementaron utilidades personalizadas para operaciones como especificación de histograma, detección de bordes y máscaras de color.

4. Programación del Sistema

Se programó cada módulo del sistema:

- **Preprocesamiento de imágenes**: Mejora del contraste mediante especificación de histograma y realce selectivo de colores para destacar líneas de la carretera.
- **Detección de carriles**: Aplicación de filtros de bordes y Hough Transform para ajustar curvas polinómicas que representen los carriles.
- **Detección de obstáculos**: Uso de un modelo YOLO preentrenado para identificar objetos como vehículos y peatones.
- **Estimación de distancias**: Cálculo aproximado con base en la posición del objeto y parámetros de la cámara.
- **Detección de esquinas**: Implementación de un detector de esquinas (Harris) para señalar posibles intersecciones o cambios de vía.

5. Integración y Visualización

Se integraron todos los módulos en un ciclo de procesamiento de vídeo, permitiendo mostrar las detecciones sobre los cuadros del video o la cámara, y facilitando la evaluación continua del desempeño del sistema.

6. Evaluación y Ajustes

Se realizaron pruebas en distintos entornos y condiciones visuales para evaluar la robustez del sistema. Con base en los resultados, se ajustaron parámetros como umbrales de color, sensibilidad de detección y regiones de interés para mejorar la precisión y la eficiencia.

Diagrama de flujo del programa

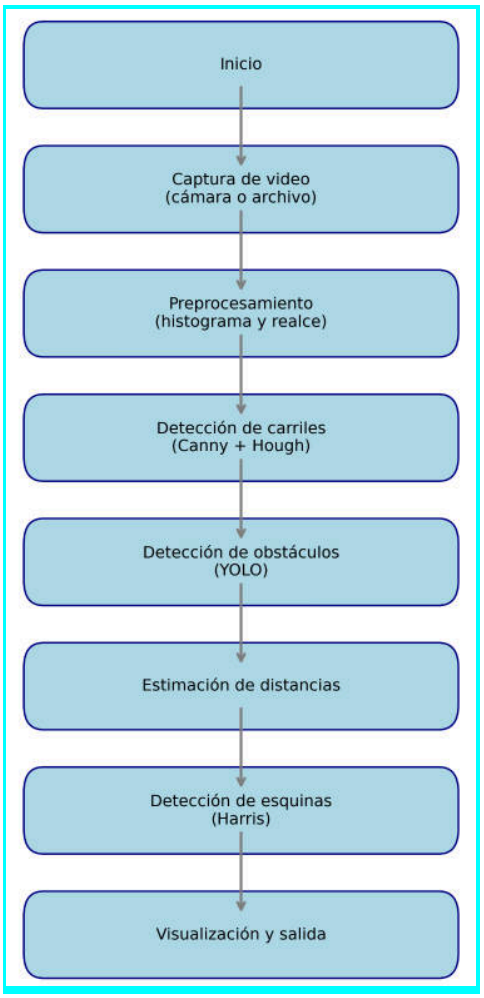


Imagen 4

Descripción General del código

main.py

Esta es la función principal del sistema. Su propósito principal es coordinar la captura, procesamiento y visualización de un flujo de video, aplicando una secuencia de módulos de análisis visual en tiempo real.

Funcionamiento:

- Captura de video: A través de cv2.VideoCapture, se obtiene un flujo continuo de imágenes desde la fuente especificada (un archivo).

Preprocesamiento:

- Detección de esquinas: Se aplica el detector de esquinas de Harris sobre una región de interés para identificar posibles intersecciones o entradas de calle.
- Detección de carriles: Utilizando un modelo basado en bordes y líneas (Canny y Hough), se identifican los límites del carril sobre la vía.
- Detección de objetos: El sistema aplica el modelo YOLOv11 para detectar peatones, vehículos u otros obstáculos presentes en la vía.
- Estimación de distancias: Se calcula una distancia aproximada entre el vehículo y los objetos detectados, con base en sus posiciones en la imagen.
- Detección de semáforos y su estado para avanzar o detenerse. Por medio de lo detectado YOLO avisa si se debe avanzar o no.
- Visualización y desempeño: Se muestra el resultado del procesamiento en tiempo real junto con el cálculo del rendimiento (FPS).

config.py

Esta clase contiene parámetros globales y constantes utilizadas a lo largo del sistema. Su propósito es centralizar la configuración de valores que afectan el comportamiento general del procesamiento. Esto nos permite darle mantenimiento de manera más sencilla.

Componentes

- **TARGET_FPS** y **FRAME_INTERVAL**: Definen la frecuencia de actualización del procesamiento de video.
- **TARGET_FPS**: Cuántos cuadros por segundo se intentan procesar (45 por defecto).
- **FRAME_INTERVAL**: Intervalo de espera entre cada frame, calculado como el inverso del FPS objetivo.
- **CLASS_NAMES**: Lista con los nombres de las clases de objetos que puede detectar el modelo YOLO. Se cargan desde el archivo coco.names, ubicado en el mismo directorio que el script.
- **COLORS_PER_CLASS**: Esta lista se utiliza para dibujar los cuadros de detección en distintos colores según el tipo de objeto (persona, auto, bicicleta, etc.).

Esta clase nos permite separar la lógica del sistema y sus parámetros.

utils.py

Esta clase reúne las funciones auxiliares necesarias para el procesamiento de imágenes y video en el sistema. Nos permite acceder a las principales operaciones que queremos aplicar y reutilizar el código.

Principales funciones:

- **draw_text**: Dibuja texto sobre una imagen con un fondo oscuro para mejorar su visibilidad, útil para superponer información como FPS o alertas.
- **fps_counter**: Generador que calcula los cuadros por segundo (FPS) en tiempo real, usando una ventana deslizante para obtener un promedio estable.
- **region_of_interest**: Aplica una máscara poligonal a la imagen para enfocarse en zonas de interés (como el camino), reduciendo ruido visual en el resto del frame.
- **auto_canny**: Implementación automática del detector de bordes Canny, ajustando los umbrales en función de la mediana de intensidades, útil para contornos adaptativos.

- **distance_to_camera**: Estima la distancia a un objeto usando su tamaño real, tamaño aparente en la imagen y distancia focal de la cámara (relación geométrica).
- **non_max_suppression**: Reduce superposiciones de múltiples detecciones de objetos (bounding boxes) reteniendo solo las más probables, común en detección con redes neuronales.
- **detect_corners_with_ro**i: Detecta esquinas relevantes solo en la parte inferior del frame (ROI), utilizando el algoritmo de Harris/Shi-Tomasi. Marca las esquinas con puntos azules, útiles para indicar posibles intersecciones o nuevas calles.

Lane_Detector.py

Esta es una clase orientada a la detección de carriles en condiciones urbanas. Utiliza ajuste polinómico, suavizado temporal y detección de líneas para trazar los límites del camino. El ajuste polinómico consiste en encontrar una curva que se aproxime a la forma del carril utilizando una función matemática, generalmente de segundo grado, a partir de los puntos detectados en la imagen. Esto permite representar carriles curvos con mayor precisión. El suavizado temporal se aplica para reducir fluctuaciones entre fotogramas consecutivos, logrando una detección más estable y continua a lo largo del tiempo. La detección de líneas se apoya en técnicas como la transformación de perspectiva y la segmentación por color o gradiente para identificar las marcas viales visibles en la carretera.

- ***Inicialización (__init__)****:* Es el constructor de la función con el cual inicializamos en caso de no agregar datos a los parámetros establecidos para que pueda trabajar el programa.
- ***detect_lanes(frame)****:* Es la función principal de detección. Convierte la imagen (frame) a escala de grises, aplica desenfoque, el detector de bordes de Canny, y limita la detección a una región de interés trapezoidal. Luego, se usan líneas de Hough para detectar segmentos, se agrupan según su pendiente en carriles izquierdo y derecho, y se ajusta un polinomio de segundo grado a cada grupo.

- **_fit_polynomial(points, height)**: Ajusta una curva polinómica de segundo grado a los puntos detectados (izquierdos o derechos) usando regresión. Elimina valores atípicos (outliers) antes del ajuste.
- **_validate_fit(fit, side)**: Verifica que la curva ajustada sea válida: dentro de los rangos definidos para los coeficientes, con una pendiente coherente (negativa para izquierda, positiva para derecha), y sin cambios bruscos respecto a cuadros anteriores.
- **_get_smooth_fit(fits)**: Calcula el promedio de los últimos ajustes válidos para suavizar las curvas y evitar parpadeos o saltos repentinos.
- **_draw_lanes(image)** y **_draw_poly_line()**: Dibuja las curvas de carril en la imagen original usando los coeficientes polinómicos suavizados. Se representa cada curva con una línea verde sobre la carretera y se marca la región de interés (ROI) con un trazo naranja claro.
- **detect_lanes(frame)**: Esta es una función externa global permite integrar fácilmente el detector con otros módulos del sistema. Manteniendo el estado interno del detector entre cuadros mediante una instancia única (singleton).

distance_estimator.py (Estimación de distancia a obstáculos)

Esta clase calcula la distancia desde la cámara del vehículo hasta los objetos detectados (vehículos, personas, motocicletas, etc.). Para lograr esto se hace una estimación en base a geometría de proyección y dimensiones conocidas de objetos. Para realizar esto, establecemos algunos parámetros estándar para los objetos. Por ejemplo:

```
self.class_heights = {
    0: 1.7,    # person (average height)
    2: 1.7,    # car (height from ground)
    5: 3.5,    # bus
    7: 3.4,    # truck
    8: 1.8,    # motorcycle (when considering rider too)
}
```

Establecemos una altura promedio de las personas, los vehículos según su tipo. Cabe aclarar que no es exacto, pero nos permite estimar un aproximado.

Finalmente clasifica los niveles de alerta según la proximidad del objeto. De esta manera el sistema tiene una aproximación de cuan cerca o lejos está un objeto y que se debería hacer después.

- **DistanceEstimator (clase principal)**: Calcula la distancia a objetos detectados.
- **Inicialización (__init__)**: Constructor de la clase.
- **camera_height, camera_pitch, focal_length**: parámetros físicos de la cámara
- **class_heights y class_widths:** alturas y anchos reales estimados por clase (persona, auto, bus, etc.).
- **warning_thresholds:** define niveles de alerta (safe, warning, critical) según distancia establecida.
- **estimate_distance(box, class_id, frame_height, frame_width):** Calcula la distancia aproximada con la información prevista. Para ello se hace uso de los siguientes parametros.
    - Posición vertical del objeto en la imagen: ángulo vertical con respecto al centro y pitch de la cámara.
    - Altura aparente en píxeles.
    - Ancho aparente en píxeles.

Despues con estos parametros se hace una media ponderada de los tres métodos, ajustando los pesos en función del tamaño del objeto en pantalla y su posición relativa (centrado o no).

Finalmente, se limita el resultado a un rango razonable (1.5–100 m) y se aplica una corrección para distancias realistas (-13.5 m). Este parámetro se puede modificar y es nuestra herramienta principal para mejorar las aproximaciones.

- **get_warning_level(distance):** Devuelve el nivel de alerta (critical, warning, safe) y su color correspondiente (rojo, naranja, verde), basado en la distancia estimada.

stop_lights.py

Esta clase está diseñada para detectar el color de semáforos y luego mostrar un mensaje en pantalla dependiendo del color detectado.

Primero se definen rangos de color en formato HSV para detectar rojo, amarillo y verde. Guarda los últimos 5 colores detectados para evitar que cambios breves (como parpadeos) cambien el mensaje rápidamente.

Después toma una región del semáforo en la imagen (una caja delimitadora) y analiza el color predominante. Calcula el color más frecuente y dando prioridad al rojo para seguridad.

Finalmente dibuja un mensaje de texto en el frame con un fondo negro y texto de color según el estado del semáforo. Dándonos el mensaje de Avance o deténgase.

yolo_detector.py

Esta clase implementa un detector de objetos basado en YOLO (You Only Look Once) optimizado para su uso en flujos de video en tiempo real. Detecta personas y vehículos y mantiene la consistencia entre cuadros, mejorando eficiencia y estabilidad visual.

Parámetros:

- model: Modelo YOLO cargado (e.g., de la librería Ultralytics).
- frame: Frame actual del video.
- frame_count: Número del frame actual (para muestreo).
- width, height: Dimensiones del frame (altura, anchura).
- last_detections: Diccionario con detecciones previas (para persistencia visual).

Para que el proyecto tuviera un mejor rendimiento, YOLO procesa cada 2 cuadros los cual nos permite reducir la carga computacional. El nivel de confianza nos permite saber la seguridad de la detección para dibujar las cajas.

¿Para qué es la persistencia visual (frames intermedios)?

En los cuadros sin detección activa, simplemente se vuelven a dibujar las últimas cajas válidas. Esto mantiene continuidad visual sin invocar el modelo constantemente.

¿Qué es YOLO y por qué se usa?

You Only Look Once (YOLO) es una arquitectura de red neuronal convolucional especializada en detectar objetos en imágenes o videos de forma rápida y precisa. Su principal característica es que realiza la detección en una instancia (one pass) de la red neuronal, lo que lo hace extremadamente rápido en comparación con otros enfoques más tradicionales como R-CNN o Fast R-CNN.

YOLO identifica y localiza objetos dentro de una imagen. Detecta las clases de los objetos (vehículos, personas, etc.). Esto se logra por medio de las coordenadas de sus cajas delimitadoras (bounding boxes). Para asegurar la clasificación de un objeto se basa en las confianzas (probabilidad de que el objeto esté realmente ahí).

YOLO nos permite aplicar un modelo robusto de manera rápida brindando:

- Velocidad: Es rápido para funcionar en tiempo real.
- Simplicidad: Trata la detección como un único problema de regresión, lo que permite un entrenamiento y despliegue más directos.
- Versatilidad: Se puede usar en múltiples aplicaciones.

La manera de operar de YOLO es la siguiente:

Primero YOLO recibe una imagen de tamaño fijo, por ejemplo: 416x416 píxeles o 640x640 píxeles. Donde procede a dividir la imagen en una rejilla (grid), por ejemplo de S x S (como 13x13, 19x19, 22x22, etc., dependiendo de la versión). Cada celda de la rejilla es responsable de detectar objetos cuyo centro cae dentro de ella.

Una vez teniendo esto cada rejilla es responsable de detectar una imagen. De esta manera tenemos las coordenadas cartesianas (x, y) las cuales son coordenadas del centro de la caja (relativas a la celda). Weight y Height (w, h): ancho y alto (relativos al tamaño de la imagen). También tenemos el parámetro de confianza (confidence score): qué tan probable es que

haya un objeto ahí y el parámetro C clases: puntuación de probabilidad para cada clase (usualmente usando softmax o sigmoid).

Teniendo como cálculos: B * (5 + C) valores (5: x, y, w, h, confianza).

Después de procesar la información tenemos de salida un tensor 3D de tamaño S x S x (B * (5 + C))

Por ejemplo:

S = 13 (rejilla 13x13)

B = 3 (3 cajas por celda)

C = 80

La salida será: 13 x 13 x 255, porque 3 * (5 + 80) = 255

Filtrado: Umbral y Non-Max Suppression (NMS)

Después de obtener todas las predicciones, se realiza un filtrado donde se eliminan las cajas con baja confianza. Se aplica Non-Maximum Suppression (NMS) para eliminar cajas que se superponen mucho y quedarse solo con la mejor.

El diagrama de la siguiente página muestra los componentes principales de la arquitectura de YOLO.

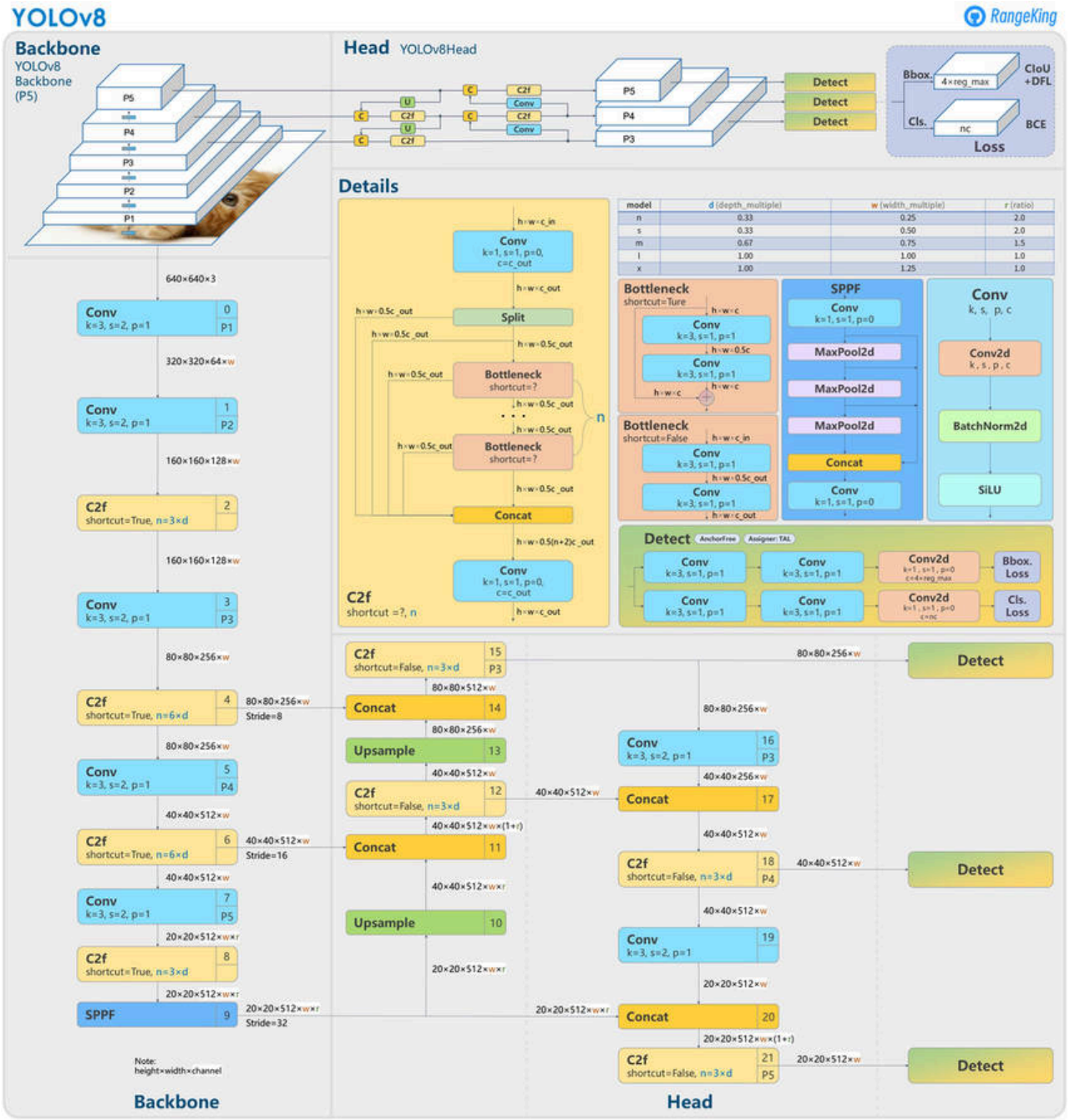


Imagen 2: Funcionamiento de las redes YOLO

Aqui se puede observar el siguiente pipeline:

- Entrada de la imagen
- Backbone: Una red neuronal convolucional extrae características de la imagen. Dependiendo la versión que se use la arquitectura puede variar como CSPDarknet.
- Neck: Son una serie de capas que combinan características de diferentes escalas para mejorar la detección de objetos de distintos tamaños.

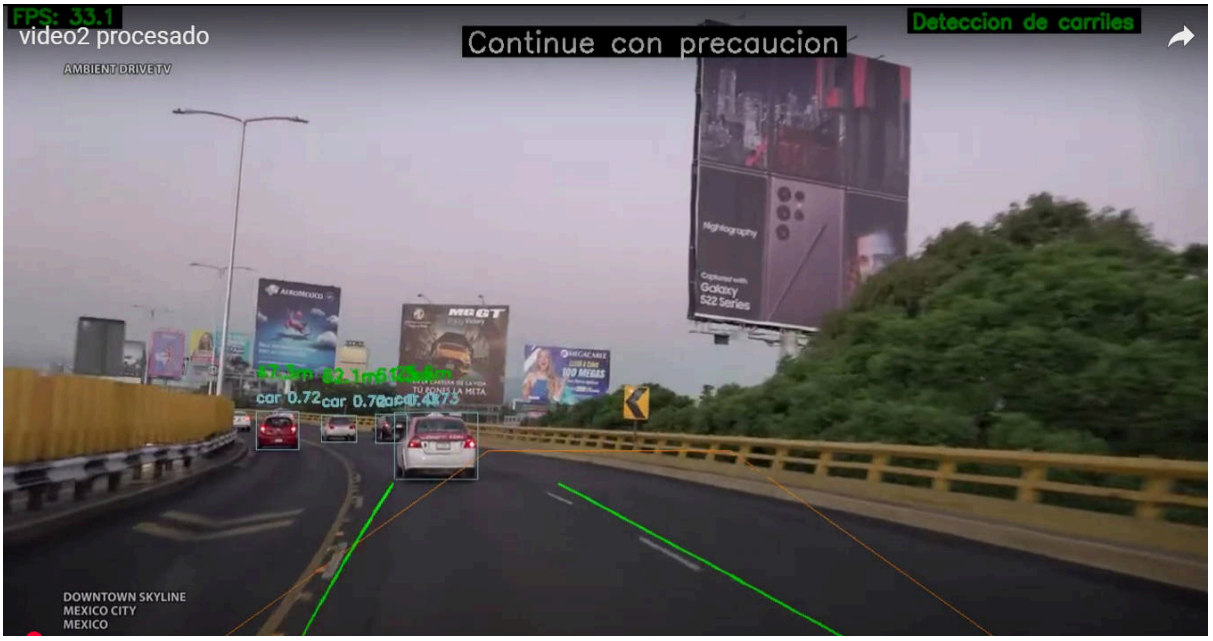
- Head: Parte de la red que realiza las predicciones de las cajas delimitadoras (bounding boxes) y las clases de los objetos.

Debido a que YOLO se entrena usando una función de pérdida (loss) que combina Error de coordenadas (x, y, w, h), error de confianza (IoU entre predicción y ground truth), error de clasificación (clase correcta del objeto) permite que todo se entrene en una sola red neuronal.

Por lo anterior mencionado hace que el proceso de detección ocurra en una sola red, en una sola instancia de la imagen haciendolo mas rapido.

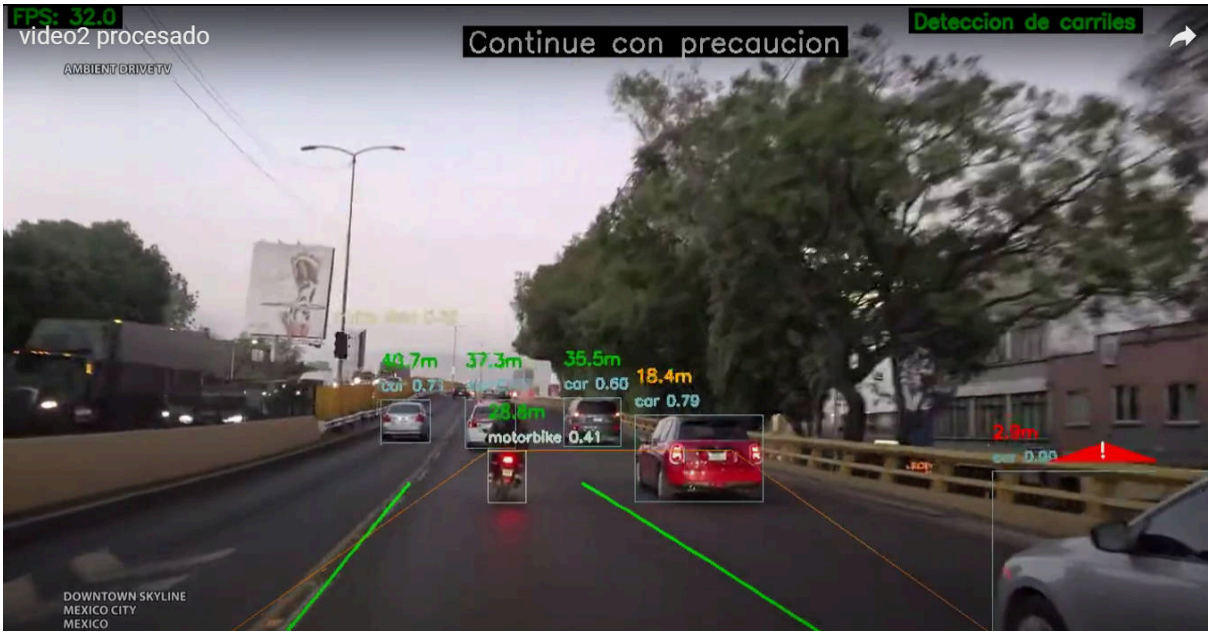
RESULTADOS

<u>Capturas de pantalla</u>




FPS: 33.5
video2 procesado
AMBIENT DRIVE TV
Continue con precaucion
Deteccion de carriles
44.8m
truck 0.36
41.7m
34.5m
car 0.44
31.5m
motorbike 0.61
5342-6667
DOWNTOWN SKYLINE
MEXICO CITY
MEXICO


FPS: 33.6
video2 procesado
AMBIENT DRIVE TV
Continue con precaucion
Deteccion de carriles
26.7m
truck 0.69
31.2m
65.8m
car 0.52
car 0.50
29.2m
motorbike 0.43
DOWNTOWN SKYLINE
MEXICO CITY
MEXICO


FPS: 32.0
video2 procesado
AMBIENT DRIVE TV
Continue con precaucion
Deteccion de carriles
37.3m
35.5m
car 0.60
18.4m
car 0.79
motorbike 0.41
2.9m
DOWNTOWN SKYLINE
MEXICO CITY
MEXICO

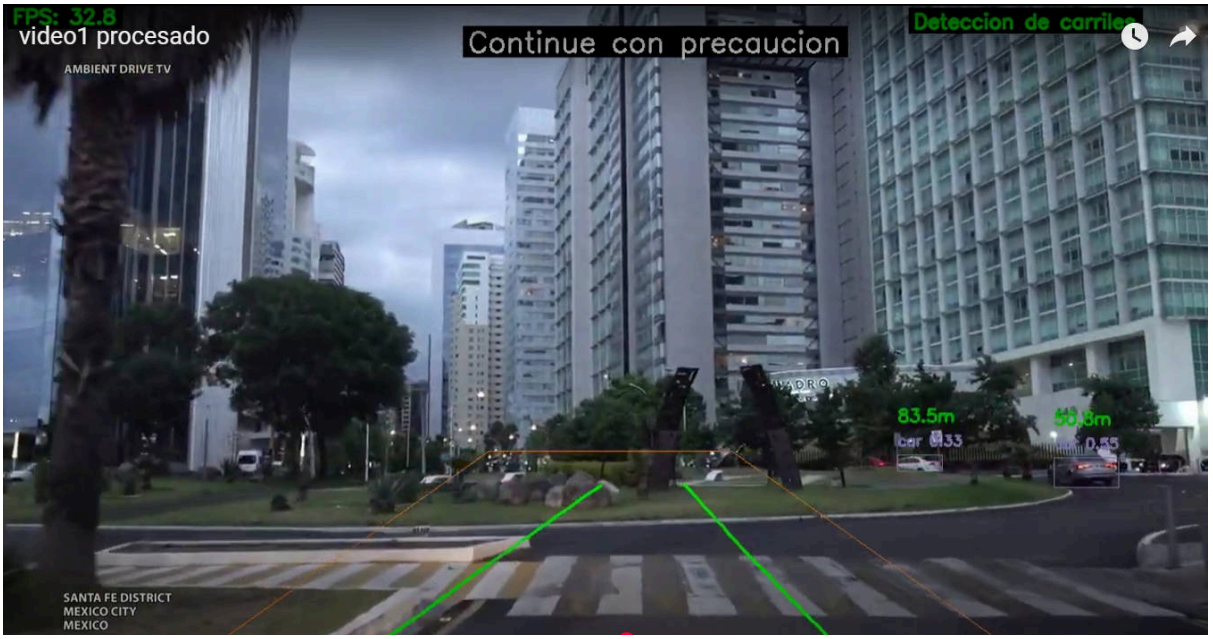

FPS: 32.9
video2 procesado
AMBIENT DRIVE TV
Continue con precaucion
Deteccion de carriles
41.4m
7.0m
car 0.88
DOWNTOWN SKYLINE
MEXICO CITY
MEXICO

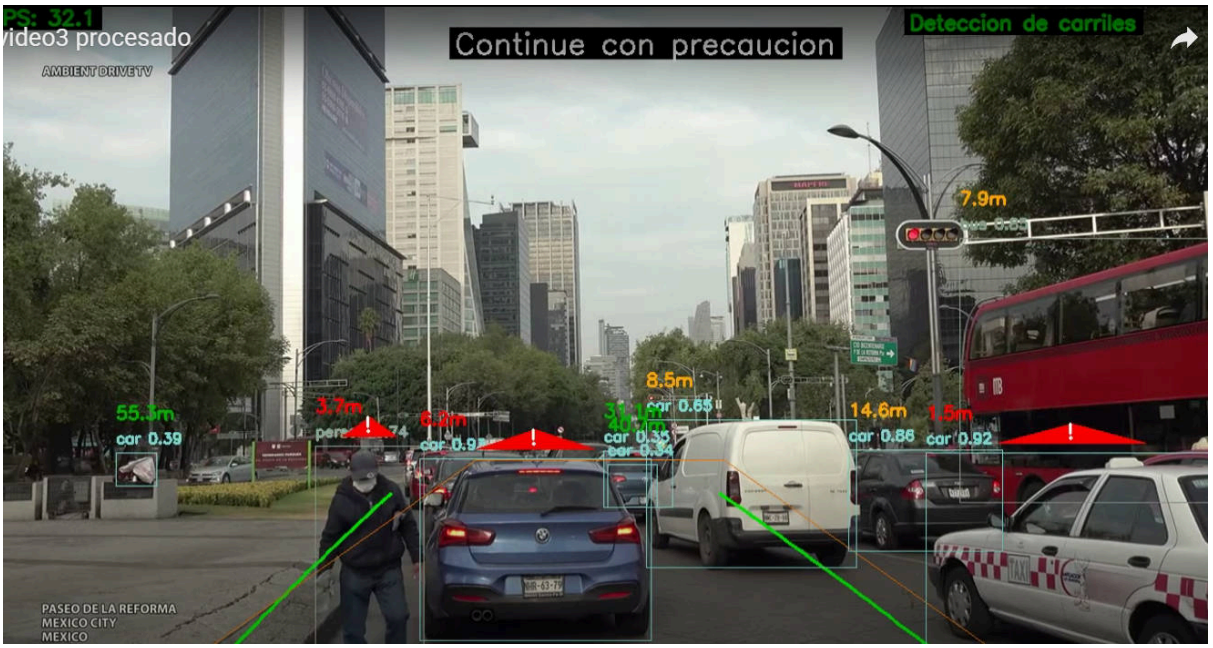

FPS: 33.3
video2 procesado
AMBIENT DRIVE TV
Continue con precaucion
Deteccion de carriles
59.1m
car 0.40
DOWNTOWN SKYLINE
MEXICO CITY
MEXICO

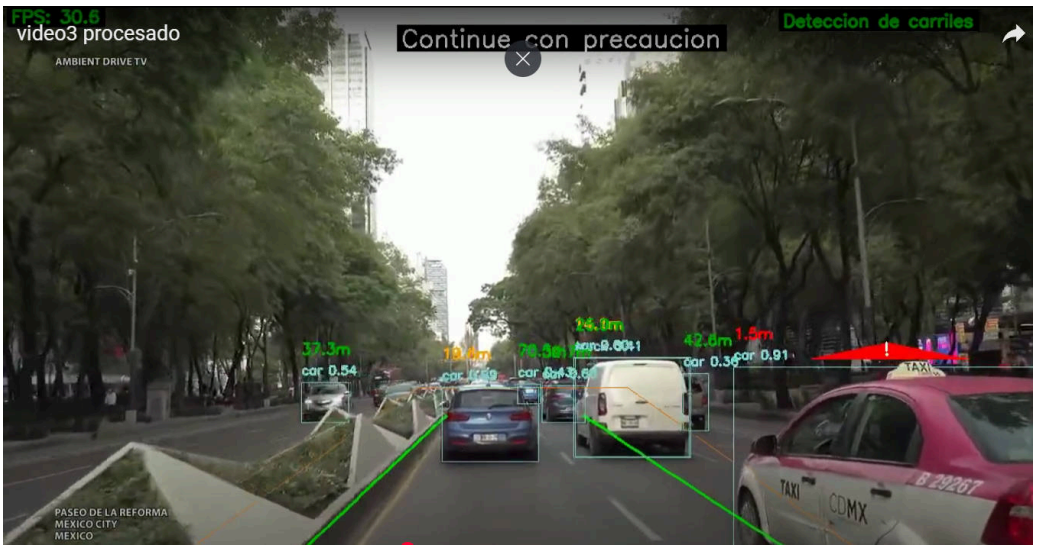

FPS: 33.2
video2 procesado
AMBIENT DRIVE TV
Continue con precaucion
Deteccion de carriles
24.8m
car 0.80
13.7m
DOWNTOWN SKYLINE
MEXICO CITY
MEXICO

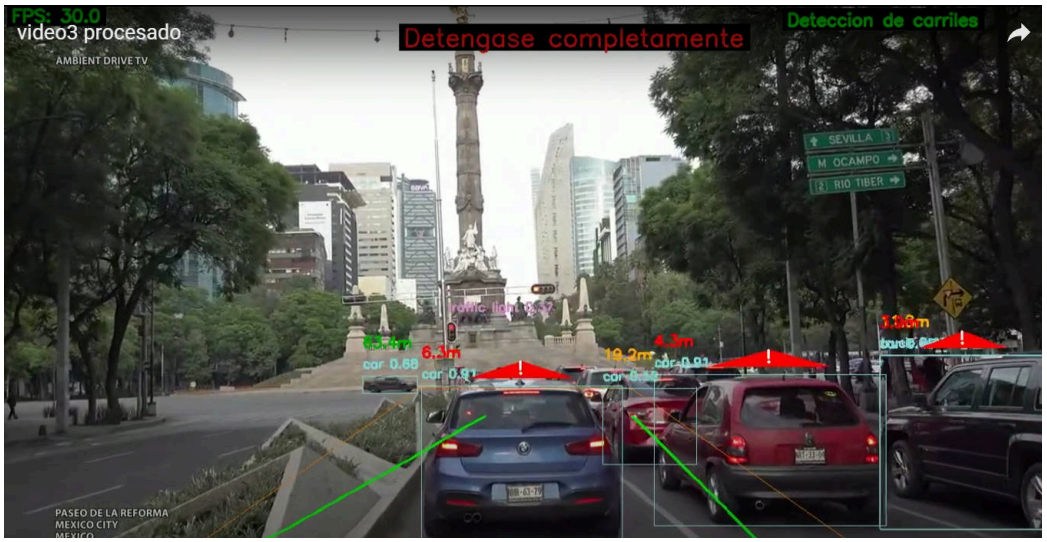

FPS: 32.2
video2 procesado
Continue con precaucion
Deteccion de carriles
AMBIENT DRIVE TV
AEROMEXICO
Nos merecemos volar
con un nuevo aire
10.4m
truck 0.82
34.5m
car 0.56
23.5m
motorbike 0.49
DOWNTOWN SKYLINE
MEXICO CITY
MEXICO


FPS: 34.1
Detengase completamente
Deteccion de carriles
AMBIENT DRIVE TV
20.5m
bus 0.46
28.2m
car 0.82
SANTA FE DISTRICT
MEXICO CITY


FPS: 32.8
video1 procesado
AMBIENT DRIVE TV
Continue con precaucion
Deteccion de carriles
83.5m
car 0.33
50.8m
SANTA FE DISTRICT
MEXICO CITY
MEXICO


PS: 32.1
ideo3 procesado
AMBIENT DRIVE TV
Continue con precaucion
Deteccion de carriles
7.9m
8.5m
car 0.65
55.3m
car 0.39
3.7m
6.2m
car 0.9
14.6m
car 0.88
1.5m
car 0.92
PASEO DE LA REFORMA
MEXICO CITY
MEXICO





Links de los videos

- Video: https://youtu.be/814QHNB8wtI
- Video: https://youtu.be/cn7wwXY7j0E
- Video: https://youtu.be/kC6ge3YLkos

Si no se visualizan en Youtube de igual manera los videos se encuentran en la siguiente dirección:

https:/ [redacted] my [redacted] com/:f:/g/personal [redacted] alumno_buap_mx/ElygtuVKQuVMt7dh7ElQ9i8BfXceRVmIxhgLrDbX4wYXgA?e=SYu8W9

CONCLUSIONES

El proyecto se implementó de forma exitosa, integrando diversas técnicas de visión por computadora para lograr una guía básica de conducción. Se emplearon métodos como Redes Neuronales Convolucionales, detección y prolongación de líneas, así como la identificación de objetos relevantes, como semáforos, para emitir recomendaciones de acción al usuario. Aunque el sistema funciona de manera adecuada, aún hay margen de mejora, especialmente en la detección de una mayor variedad de objetos y en la toma de decisiones más complejas.

Algunos de los problemas principales son la calidad de los vídeos afectan al modelo para detectar bien los objetos. Así mismo, el cambio de condiciones afectan el rendimiento del modelo en cómo detecta. Se intentó con una especificación del histograma para homogeneizar las imágenes, sin embargo esto tuvo como efecto colateral la ralentización del modelo sin mejoras significativas. Aún deben explorarse más opciones aquí.

A pesar de ello, los resultados obtenidos son satisfactorios y demuestran el potencial de este enfoque.

TRABAJO FUTURO

Para continuar con el desarrollo de este sistema, existen varias líneas de mejora que podrían explorarse. Una de las más importantes es la ampliación de la capacidad de detección, incorporando más clases de objetos relevantes para la conducción, como peatones, señales de tránsito, ciclistas o vehículos detenidos manteniendo el sistema ligero. Asimismo, se podrían implementar algoritmos de toma de decisiones más avanzados que permitan reaccionar ante situaciones más complejas del entorno. Finalmente, integrar técnicas de aprendizaje continuo permitiría que el modelo mejore su precisión con el tiempo a partir de nuevas experiencias.

BIBLIOGRAFÍA

- *Andigital. (2023, septiembre 19). El 90% de los accidentes viales son causa del error humano. Andigital. https://andigital.com.ar/nota/119727/el-90-de-los-accidentes-viales-son-causa-del-error-humano/*
- *El Economista. (2023, 4 de junio). Crecen accidentes viales; protege tus finanzas de ellos. El Economista. https://www.eleconomista.com.mx/finanzaspersonales/Crecen-accidentes-viales-protege-tus-finanzas-de-ellos-20230604-0073.html*
- *El Economista. (2010, noviembre 16). Por fallas humanas, 70% de accidentes. El Economista. https://www.eleconomista.com.mx/finanzaspersonales/Por-fallas-humanas-70-de-accidentes-20101116-0068.html*
- *Todo Riesgo. (2023, septiembre 19). El 90% de los accidentes de tránsito es producto del error humano. Todo Riesgo. https://www.todoriesgo.com.ar/el-90-de-los-accidentes-de-transito-es-producto-del-error-humano/*
- *Organización Mundial de la Salud. (2023). Seguridad vial. https://www.who.int/es/news-room/fact-sheets/detail/road-traffic-injuries*
- *RangeKing. (2024). NFE-YOLO: A Lightweight and Efficient Detection Network for Low, Slow, and Small Drones. ResearchGate. https://www.researchgate.net/figure/YOLOv8-Architecture-Diagram-Including-Backbone-Neck-and-Head-Units_fig4_3855720*